

HwOS 1.1 Technical Report: Ownership, Functional Hardware, and Zero-Bubble Concurrency

Author: Kaixin Chen (Zhejiang University)

Date: [2026/2/27]

Project Repository: [<https://github.com/NullStarfish/HwOS>]

Relation: This report is a major architectural update and philosophical refinement to the HwOS 1.0 Concept (DOI: [<https://doi.org/10.36227/techrxiv.177155627.77438450/v1>]).

1. Introduction: The Evolution of TL-RTL Philosophy

In HwOS 1.0, we introduced the concept of **Thread-Level RTL (TL-RTL)**, transforming fragmented Finite State Machines (FSMs) into a unified Thread-Process architecture. However, to truly elevate hardware design to the level of modern software engineering, we must address the fundamental philosophies of resource ownership, logic decoupling, and precise timing control.

HwOS 1.1 is not merely a feature update; it is a paradigm shift. We have deprecated explicit `PhysicalDriver` boundaries, completely decoupled execution containers from logic payloads via `HwFunction`, and introduced OS-level "Garbage Collection" for hardware states. This report outlines the core mechanisms that make HwOS 1.1 a safe, high-performance, and functional hardware operating system.

2. Ownership: From Explicit Drivers to Bottom-Up Agents

In traditional hardware frameworks, shared resources are often encapsulated within rigid, explicit class definitions (like `PhysicalDriver` in HwOS 1.0). In 1.1, we abandoned this heavy abstraction in favor of a bottom-up **Ownership Model**.

All hardware logic and resource management are now localized within `HardwareAgent` (which includes `HardwareThread` and `HardwareLogic`).

- **The Philosophy:** Resources (Wires/Registers) do not belong to abstract "Drivers"; they physically belong to the `Agent` that instantiates them.
- **The Implementation:** We introduced `HwOwner`. An agent must explicitly `own` a signal and `grant` permissions to other agents via a strict Access Control List (ACL). The `<==` operator physically gates assignments based on the agent's active state.

```
// Example: Ownership and ACL Granting in HwProcess
val resultReg = RegInit(0.U(32.W))
this.own(resultReg)           // Process owns the register
this.grant(resultReg, consumer) // Grants write access to consumer thread

consumer.entry {
    consumer.Step("Write") {
        resultReg <== 123.U    // Safe assignment: dynamically gated by consumer's isActive
    }
}
```

3. HwFunction: Decoupling the "CPU" from the "Program"

A fundamental flaw in traditional RTL FSMs is the tight coupling of state machines (the execution container) and the actual datapath logic. HwOS 1.1 completely decouples them by introducing `HwFunction`.

- **The Analogy:** If a `HardwareThread` is the **CPU** (providing the PC, execution context, and lifecycle), then `HwFunction` is the **Program Code** (the stateless logic payload injected into the CPU).
- **Functional Hardware Era:** By introducing `SysCall` and `CallStack`, hardware development enters the functional era. Hardware logic can now be passed, nested, and called just like software functions, generating a clear Debugging Call Stack for complex RTL.

```
// Defining a reusable hardware function (The "Program")
def GuardedRead(addr: UInt): HwFunction[UInt] = HwFunction.atomic("GuardedRead") { t =>
    val ready = SysCall.Call(scoreboard.Guard(addr)) // Nested SysCall
    val rdata = this.own(WireInit(0.U(32.W)))
    grant(rdata, t)

    when(ready) { rdata <== baseReg.Read(addr) }
    rdata // Returns the hardware artifact
}

// Executing it inside a Thread (The "CPU")
consumer.Step("ReadOperand") {
    // SysCall pushes "GuardedRead" to the CallStack for semantic debugging
    val data = SysCall.Call(GuardedRead(5.U))
}
```

4. HwLease & OS Reaper: System-Level Garbage Collection

In HwOS 1.0, the `abort()` mechanism could clear transient wire states by cutting off the thread's active signal. However, **stateful resources** (e.g., Registers representing Mutex locks or Scoreboard busy-bits) would remain permanently locked if a thread died, leading to unrecoverable system deadlocks.

To solve this, HwOS 1.1 introduces `HwLease` —a system-level contract.

- **Stateful Resource Recovery:** When a thread acquires a lock, the `HwLease` is registered directly into the thread's PCB (`HwContext`).
- **The OS Reaper:** If a thread is killed (e.g., `SysCall.kill(target)`), the OS Reaper takes over. It bypasses user-space ACLs and uses the privileged operator `<==!` to trigger `forceReclaim` , acting as a system-level **Garbage Collector** for hardware locks.

```
// Inside a MutexLease
override def forceReclaim(agent: HardwareAgent): Unit = {
  // Privileged assignment (<==!) to forcibly tear down the contract
  // even if the thread's user-space context is already dead.
  unlocks(id) <==! true.B
  isHeld      <==! false.B
}
```

5. Hijack: Compilation-Level Zero-Bubble Timing

High-level hardware abstractions often suffer from degraded timing control (e.g., unavoidable 1-cycle FSM handshake bubbles). HwOS 1.1 resolves this via the `Next.hijack()` mechanism.

- **The Philosophy:** The user writes code in a highly abstract, sequential manner (Step A -> Step B). However, at the compilation level, `hijack` allows Step A to dynamically "devour" and inline the combinational closure of Step B.
- **Invisible Optimization:** The user doesn't feel the existence of the hijack in their source code's logical flow, but the physical performance realizes a Zero-Bubble, 0-cycle transition.

```
consumer.Step("WaitIssue") {
  consumer.waitCondition(flagReserved)
  when(flagReserved) {
    consumer.Next.hijack() // Hijacks the next step immediately!
  }
}
consumer.Step("ReadOperand") { // This logic is inlined into the previous cycle
  resultReg <== SysCall.Call(regfile.GuardedRead(5.U))
}
```

To fully appreciate the elegance of `Next.hijack()`, it is crucial to understand that it is **not a runtime hardware behavior**, but a compiler-level metaprogramming directive evaluated during Chisel's elaboration phase.

In traditional Verilog, "skipping" a state or transitioning in zero cycles usually requires writing complex, deeply nested combinational logic (e.g., bypassing registers with multiplexers). In HwOS 1.1, `hijack` achieves this without touching the runtime datapath overhead.

- **The Mechanism:** When `Next.hijack()` is invoked, the HwOS compiler intercepts the AST (Abstract Syntax Tree) generation. It flags the subsequent `StepNode` as `isHijacked = true` and immediately executes the victim step's Scala closure (`victim.block()`) **inline** within the current step's context.
- **The Hardware Result:** Because the victim step is marked as hijacked, the HwOS FSM generator completely ignores it when allocating Program Counter (PC) addresses. The synthesizer never sees the victim step as a distinct FSM state. Instead, the logic of both steps is fused into a single massive combinational block assigned to a single physical PC.

```
// Under the hood in HardwareAgent.scala (Elaboration Time)
def hijack(): Unit = {
    val victim = currentGeneratingNode.next
    victim.isHijacked = true // 1. Shield the victim from PC allocation

    // 2. Inline the victim's hardware generation closure into the current PC's 'when' block
    ContextScope.withContext(AtomicCtx(HardwareThread.this)) {
        victim.block()
    }
}
```

6. The Hardware Stdlib & Pluggable Datapaths

To prove the robustness of the Ownership and HwFunction abstractions, HwOS 1.1 ships with a comprehensive standard library (`HwOS.stdlib`).

- **Concurrency Primitives:** `Mutex` , `Semaphore` , and a highly optimized `Scoreboard` .
- **Go-Inspired Synchronization:** `WaitGroup` (using concurrent adder-trees and `PopCount` to prevent same-cycle data stomping) and `Select` .

Case Study: Highly Decoupled ScoreboardRegfile

We demonstrated that scheduling logic is highly structured, pluggable, and completely decoupled from the physical datapath. We took a raw, pure-datapath `BaseRegfileProcess` and seamlessly merged it with a `ScoreboardProcess` (from the Stdlib) to create a `ScoreboardRegfileProcess` .

```

class ScoreboardRegfileProcess(...) extends HwProcess {
  // 1. Pure Datapath
  val baseReg = spawn(new BaseRegfileProcess(...))
  // 2. Pure Control/Scheduling Logic (from Stdlib)
  val scoreboard = spawn(new ScoreboardProcess(...))

  // 3. Merging them via HwFunction
  class RegWritePort(val portIdx: Int) {
    def WritebackAndClear(addr: UInt, data: UInt): HwFunction[Unit] = HwFunction.stateless(
      val sbLease = SysCall.Call(scoreboard.RequestLease(portIdx))
      SysCall.Call(baseReg.Write(portIdx, addr, data)) // Datapath action
      SysCall.Call(sbLease.Release())                 // Control action
    )
  }
}

```

This elegant composition flawlessly resolves RAW and WAW hazards without tainting the underlying memory arrays with scheduling logic.

7. System Validation & Conclusion

We rigorously tested the `ScoreboardRegfile` for pipeline stall correctness, the `Stdlib` for concurrency limits, and the `Mutex` for OS-level resource recovery (`MutexAbortSpec`). The system gracefully handled dynamic thread terminations, recovered all stateful locks via `HwLease` , and forwarded data with zero bubbles via `hijack` .

HwOS 1.1 proves that introducing deep OS philosophies (Ownership, System Calls, Garbage Collection) into RTL design does not compromise hardware performance. Instead, it unlocks unprecedented composability and safety for the next generation of agile hardware development.

8. Native Observability: Dual-Track CallStack and HwOSgdb

The Dual-Track CallStack

Because HwOS decouples the CPU (`HardwareThread`) from the Program (`HwFunction`), traditional flattened PC tracking is insufficient. We implemented a `ThreadLocal callStack` tracker during the elaboration phase to capture a **Dual-Track execution trace**:

1. **Temporal Stack (Macro)**: Tracks the nested depth of time-consuming `HwFunction.thread` calls (e.g., `NetworkTX -> SendPayload`).
2. **Atomic Tree (Micro)**: Tracks the combinational, unrolled logic paths within a single cycle (e.g., inside a `Step` , tracking `SysCall.Call(StoreData(HashData(X)))`).

The Kernel dumps this rich, multi-dimensional execution graph into a static symbol table (`hwos.symbols`).

DPI-C Telemetry and TUI Debugger

To bridge the simulation and the symbols, the HwOS Kernel automatically injects a `KernelStateMonitorDPI` . This zero-intrusion hardware probe aggregates the `pc` , `active` , and `done` signals of every registered thread, packing them into a wide vector.

At every clock cycle, the SystemVerilog DPI-C interface (`kernel_monitor_tick`) streams this telemetry to a C++ Verilator backend. The accompanying **HwOSgdb** (built with `ncurses`) cross-references this live telemetry with `hwos.symbols` to provide:

- **Live Semantic TUI**: Visualizing executing threads, hijacked steps, and their precise `callStack` traces in real-time.
- **Time-Travel Debugging**: A scrolling History Buffer (`cycleSnapshot`) that allows developers to step backward through cycle ticks to pinpoint the exact moment an `HwLease` deadlock or RAW hazard occurred.